

MATH70102 UNSUPERVISED LEARNING

LIVE SESSION — WEEK 2

Mikko Pakkanen

Department of Mathematics
Imperial College London

Summer 2026

IMPERIAL

Review of this week's content

Computational examples

Problem sheet questions

Principal component analysis

Overview

- Fundamentally, a way to transform multivariate data into **orthogonal principal components** with decreasing variance
- Becomes a **dimensionality reduction method** once we **omit** some of the principal components with lowest variance
- Can be shown to give **optimal, linear lower-dimensional description** of the data (**reconstruction error minimisation**)

Principal component analysis (cont'd)

Data and basic sample statistics

- **Samples:** $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$
- **Sample mean:** $\bar{\mathbf{x}} := \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$
- **Sample covariance matrix:**

$$\Sigma := \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

(sometimes normalisation $\frac{1}{n-1}$ is used)

Principal component analysis (cont'd)

k -th principal component

- **Projection** via vector $\mathbf{c}_k \in \mathbb{R}^d$
- k -th principal component: $\mathbf{c}_k^T(\mathbf{x}_i - \bar{\mathbf{x}})$, $i = 1, \dots, n$
- **Maximise** sample variance of projected data

$$V(\mathbf{c}_k) = \mathbf{c}_k^T \Sigma \mathbf{c}_k$$

such that

$$\underbrace{\mathbf{c}_k^T \mathbf{c}_k = 1}_{\text{normalisation}} \quad \underbrace{\mathbf{c}_k^T \mathbf{c}_j = 0 \text{ for } j < k}_{\text{orthogonality}}$$

Principal component analysis (cont'd)

PCA solution

- $\mathbf{c}_k$ is the k -th **eigenvector** of Σ ,

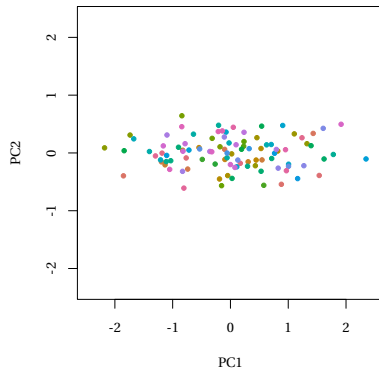
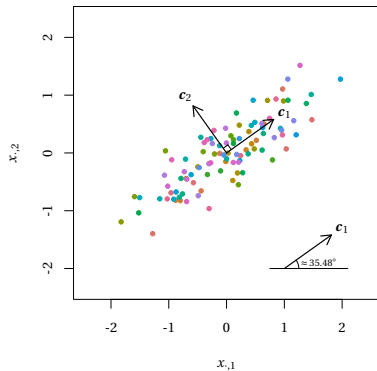
$$\Sigma \mathbf{c}_k = \lambda_k \mathbf{c}_k,$$

where λ_k is the k -th eigenvalue

- Principal components are **ordered according to variance**:

$$\underbrace{V(\mathbf{c}_1)}_{=\lambda_1} \geq \underbrace{V(\mathbf{c}_2)}_{=\lambda_2} \geq \cdots \geq \underbrace{V(\mathbf{c}_d)}_{=\lambda_d}$$

Principal component analysis (cont'd)



Principal component analysis (cont'd)

Reconstruction error minimisation

- Fix $q \leq d$ and minimise **reconstruction error**

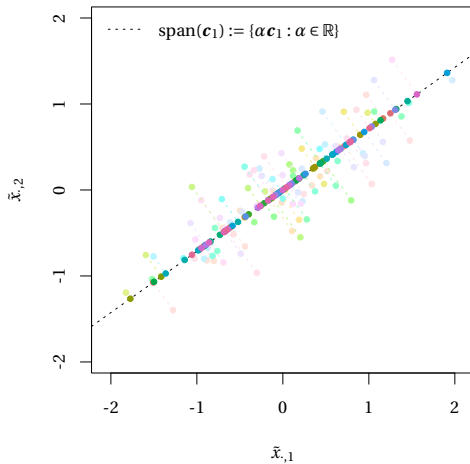
$$\frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu} - U_q \mathbf{z}_i\|^2$$

over $\boldsymbol{\mu} \in \mathbb{R}^d$, $U_q \in \mathbb{R}^{d \times q}$ such that $U_q^T U_q = I_q$ and $\mathbf{z}_1, \dots, \mathbf{z}_n \in \mathbb{R}^q$

- Solution:**

$$\boldsymbol{\mu} = \bar{\mathbf{x}}, \quad U_q = [\mathbf{c}_1 \quad \cdots \quad \mathbf{c}_q], \quad \mathbf{z}_i = U_q^T (\mathbf{x}_i - \bar{\mathbf{x}}), \quad i = 1, \dots, n$$

Principal component analysis (cont'd)



Sparse principal component analysis

Overview

- In ordinary PCA, each principal component is a linear combination of **all input variables**
- May make it hard to **interpret** principal components
- Hence, it would be useful to have **sparse** loadings, i.e., many (or most) of the components $\mathbf{c}_k$ **equal to zero**
- **Sparse PCA** constructs sparse principal components by combined **L^1/L^2 penalisation** (similar to **elastic net**)

Sparse principal component analysis (cont'd)

Norms

- For $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$,

$$\|\mathbf{x}\| := \|\mathbf{x}\|_2 := \sqrt{x_1^2 + \dots + x_d^2} \quad (L^2/\text{Euclidean norm})$$

$$\|\mathbf{x}\|_1 := |x_1| + \dots + |x_d| \quad (L^1 \text{ norm})$$

- In supervised learning, L^1 norm is used for **Lasso** and L^2 for **ridge regression**

Sparse principal component analysis (cont'd)

Reconstruction error with penalty

- Assume $\bar{\mathbf{x}} = \mathbf{o}$, fix $q \leq d$ and define

$$F(\Theta, U) := \sum_{i=1}^n \|\mathbf{x}_i - \Theta U^T \mathbf{x}_i\|^2 + \underbrace{\lambda \sum_{j=1}^q \|\mathbf{u}_j\|_2^2}_{L^2 \text{ penalty}} + \underbrace{\sum_{j=1}^q \lambda_{1,j} \|\mathbf{u}_j\|_1}_{L^1 \text{ penalty}}$$

where $\Theta \in \mathbb{R}^{d \times q}$ is such that $\Theta^T \Theta = I_q$ and
 $U := [\mathbf{u}_1 \quad \cdots \quad \mathbf{u}_q] \in \mathbb{R}^{d \times q}$

- $\lambda, \lambda_{1,1}, \dots, \lambda_{1,q} \geq 0$ are **tuning parameters**

Sparse principal component analysis (cont'd)

Solution

- The error $F(\Theta, U)$ **cannot** be minimised analytically
- Instead, we use numerical algorithms to **alternatingly** minimise $F(\Theta, U)$ with respect to Θ and U until **convergence** is achieved
- Notably, minimisation with respect to Θ boils down to minimising

$$\sum_{i=1}^n \|\mathbf{x}_i - \Theta U^T \mathbf{x}_i\|^2$$

subject to $\Theta^T \Theta = I_q$, which can be solved using **singular value decomposition**

Sparse principal component analysis (cont'd)

Remark

- We did **not** assume that the columns $\mathbf{u}_1, \dots, \mathbf{u}_q$ of U are **orthogonal**
- Hence the sparse PCs are typically **correlated** (unlike ordinary PCs)

Kernel principal component analysis

Overview

- Ordinary PCA may be **ineffective** if the data exhibit **non-linear** dependence structures
- PCA need not be abandoned altogether — we can try to **transform** the data by a **feature map**, then apply PCA
- Applying PCA in the possibly high-dimensional **feature space** may be computationally costly
- We can sidestep this issue by applying the **kernel trick**

Kernel principal component analysis (cont'd)

Kernel PCA formulation

- We work with samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and feature map $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^p$
- **Apply PCA** to the transformed samples $\mathbf{f}(\mathbf{x}_1), \dots, \mathbf{f}(\mathbf{x}_n)$ **in the feature space** $\mathbb{R}^p$
- For simplicity, assume $\frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i) = \mathbf{0}$ (**this can be relaxed**)
- The PCs are found by solving an **eigenvalue problem**

Kernel principal component analysis (cont'd)

Kernel trick

- The eigenvalue problem and projection on to kernel PCs depends on the feature map $\mathbf{f}$ through **scalar products**

$$\mathbf{f}(\mathbf{x})^T \mathbf{f}(\mathbf{y}) \in \mathbb{R}, \quad \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

- Instead of working with **explicit** feature map $\mathbf{f}$, we specify it **implicitly** by a kernel $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ via

$$\mathbf{f}(\mathbf{x})^T \mathbf{f}(\mathbf{y}) := k(\mathbf{x}, \mathbf{y})$$

- This is the **kernel trick**

Kernel principal component analysis (cont'd)

Standard kernels

- **Polynomial:** $k(\mathbf{x}, \mathbf{y}) := (\mathbf{x}^T \mathbf{y} + c_0)^r$
- **Radial basis function:** $k(\mathbf{x}, \mathbf{y}) := \exp(\gamma \|\mathbf{x} - \mathbf{y}\|^2)$
- **Sigmoid:** $k(\mathbf{x}, \mathbf{y}) := \tanh(\gamma \mathbf{x}^T \mathbf{y} + c)$

Remark

- Kernel PCA is useful for **feature construction**
- But generally we **cannot reconstruct** data in the original space $\mathbb{R}^d$ from kernel PCs in the feature space $\mathbb{R}^p$

Autoencoder

Overview

- Non-linear extension of PCA based on reconstruction error minimisation
- Introduce feedforward neural networks $\mathbf{E} : \mathbb{R}^d \rightarrow \mathbb{R}^q$ (encoder) and $\mathbf{D} : \mathbb{R}^q \rightarrow \mathbb{R}^d$ (decoder) for q much smaller than d
- Train $\mathbf{E}$ and $\mathbf{D}$ to minimise reconstruction error

$$\sum_{i=1}^n \|\mathbf{x}_i - \mathbf{D}(\mathbf{E}(\mathbf{x}_i))\|^2$$

using standard deep learning techniques

Review of this week's content

Computational examples

Problem sheet questions

Review of this week's content

Computational examples

Problem sheet questions

Problem 1

Question

Show that, under ordinary PCA, the principal components $\mathbf{c}_j^T(\mathbf{x}_i - \bar{\mathbf{x}})$, $i = 1, \dots, n$, and $\mathbf{c}_k^T(\mathbf{x}_i - \bar{\mathbf{x}})$, $i = 1, \dots, n$, are **uncorrelated** for $j \neq k$ in the sense that their sample correlation is zero.

Problem 1

Solution

It suffices to show that **sample covariance is zero**:

$$\begin{aligned}
 \frac{1}{n} \sum_{i=1}^n \mathbf{c}_j^T (\mathbf{x}_i - \bar{\mathbf{x}}) \mathbf{c}_k^T (\mathbf{x}_i - \bar{\mathbf{x}}) &= \frac{1}{n} \sum_{i=1}^n \mathbf{c}_j^T (\mathbf{x}_i - \bar{\mathbf{x}}) (\mathbf{c}_k^T (\mathbf{x}_i - \bar{\mathbf{x}}))^T \\
 &= \mathbf{c}_j^T \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}}) (\mathbf{x}_i - \bar{\mathbf{x}})^T \mathbf{c}_k \\
 &= \mathbf{c}_j^T \Sigma \mathbf{c}_k = \mathbf{c}_j^T (\lambda_k \mathbf{c}_k) = \lambda_k \mathbf{c}_j^T \mathbf{c}_k = 0
 \end{aligned}$$

since $\mathbf{c}_k$ is an eigenvector of Σ with eigenvalue λ_k

Problem 2

Question

Under ordinary PCA, the loadings vector $\mathbf{c}_2$ of the second PC maximises

$$V(\mathbf{c}_2) = \mathbf{c}_2^T \Sigma \mathbf{c}_2 \quad \text{such that} \quad \mathbf{c}_2^T \mathbf{c}_2 = 1 \quad \text{and} \quad \mathbf{c}_2^T \mathbf{c}_1 = 0,$$

where Σ is the sample covariance matrix of the input data and $\mathbf{c}_1$ is the loadings vector of the first PC. Form the **Lagrangian function** of this maximisation problem and show that $\mathbf{c}_2$ is an **eigenvector** of Σ .

Problem 2

Solution

Lagrangian:

$$\mathcal{L}(\mathbf{c}_2, \lambda_2, \tilde{\lambda}) := \mathbf{c}_2^T \Sigma \mathbf{c}_2 + \lambda_2 (1 - \mathbf{c}_2^T \mathbf{c}_2) + \tilde{\lambda} \mathbf{c}_2^T \mathbf{c}_1$$

Maximiser $(\mathbf{c}_2, \lambda_2, \tilde{\lambda})$ must satisfy $\nabla_{\mathbf{c}} \mathcal{L}(\mathbf{c}_2, \lambda_2, \tilde{\lambda}) = \mathbf{0}$, whereby

$$2\Sigma \mathbf{c}_2 - 2\lambda_2 \mathbf{c}_2 + \tilde{\lambda} \mathbf{c}_1 = \mathbf{0}$$

Problem 2

Solution (cont'd)

Pre-multiplying both sides by $\mathbf{c}_1^T$ we get

$$2\mathbf{c}_1^T \Sigma \mathbf{c}_2 - 2\lambda_2 \underbrace{\mathbf{c}_1^T \mathbf{c}_2}_{=0} + \tilde{\lambda} \underbrace{\mathbf{c}_1^T \mathbf{c}_1}_{=1} = \underbrace{\mathbf{c}_1^T \mathbf{0}}_{=0},$$

where

$$\mathbf{c}_1^T \Sigma \mathbf{c}_2 = (\Sigma \mathbf{c}_1)^T \mathbf{c}_2 = \lambda_1 \mathbf{c}_1^T \mathbf{c}_2 = 0$$

since $\mathbf{c}_1$ is an eigenvector of Σ with eigenvalue λ_1

Problem 2

Solution (cont'd)

This indicates that $\tilde{\lambda} = 0$, so that

$$2\Sigma\mathbf{c}_2 - 2\lambda_2\mathbf{c}_2 = \mathbf{0}$$

Hence,

$$\Sigma\mathbf{c}_2 = \lambda_2\mathbf{c}_2$$

Problem 3

Question

Find a feature map $\mathbf{f} : \mathbb{R} \rightarrow \mathbb{R}^p$ (for some $p \in \mathbb{N}$) such that the quadratic kernel

$$k(x, y) := (xy + c)^2, \quad x, y \in \mathbb{R},$$

where $c \in \mathbb{R}$ is a constant, satisfies

$$\mathbf{f}(x)^T \mathbf{f}(y) = k(x, y), \quad x, y \in \mathbb{R}.$$

Demonstrate (by an example, say) that the feature map is not unique.

Problem 3

Solution

Firstly, note that

$$\begin{aligned}k(x, y) &= (xy + c)^2 = x^2y^2 + 2xyc + c^2 \\&= x^2y^2 + \sqrt{2cx}\sqrt{2cy} + c^2 \\&= (x^2, \sqrt{2cx}, c)^T (y^2, \sqrt{2cy}, c)\end{aligned}$$

Hence,

$$k(x, y) = \mathbf{f}(x)^T \mathbf{f}(y)$$

holds with $\mathbf{f} : \mathbb{R} \rightarrow \mathbb{R}^3$ such that

$$\mathbf{f}(x) := (x^2, \sqrt{2cx}, c)$$

Problem 3

Solution (cont'd)

Why not unique: $k(x, y) = \tilde{\mathbf{f}}(x)^T \tilde{\mathbf{f}}(y)$ holds also with

$$\tilde{\mathbf{f}}(x) := (\sqrt{2cx}, x^2, c)$$

More generally, if $A \in \mathbb{R}^{3 \times 3}$ is **orthogonal** ($A^T A = I_3 = A A^T$), then

$$\hat{\mathbf{f}}(x) := A \mathbf{f}(x), \quad x \in \mathbb{R}$$

satisfies

$$\begin{aligned} \hat{\mathbf{f}}(x)^T \hat{\mathbf{f}}(y) &= (A \mathbf{f}(x))^T A \mathbf{f}(y) \\ &= \mathbf{f}(x)^T A^T A \mathbf{f}(y) \\ &= \mathbf{f}(x)^T I_3 \mathbf{f}(y) = \mathbf{f}(x)^T \mathbf{f}(y) = k(x, y) \end{aligned}$$

Problem 4

Question

Derive equation (1.15) in the lecture notes from equation (1.14) therein.

Problem 4

Solution

Equation (1.14) reads $\mathbf{f}(\mathbf{x}_i)^T \Sigma_{\mathbf{f}} \mathbf{c} = \lambda \mathbf{f}(\mathbf{x}_i)^T \mathbf{c}$ for any $i = 1, \dots, n$

Substituting $\mathbf{c} = \sum_{j=1}^n \alpha_j \mathbf{f}(\mathbf{x}_j)$ and the definition of $\Sigma_{\mathbf{f}}$, the left-hand side becomes

$$\begin{aligned}
 \mathbf{f}(\mathbf{x}_i)^T \frac{1}{n} \sum_{k=1}^n \mathbf{f}(\mathbf{x}_k) \mathbf{f}(\mathbf{x}_k)^T \sum_{j=1}^n \alpha_j \mathbf{f}(\mathbf{x}_j) \\
 = \frac{1}{n} \sum_{j=1}^n \sum_{k=1}^n \underbrace{\mathbf{f}(\mathbf{x}_i)^T \mathbf{f}(\mathbf{x}_k)}_{K_{i,k}} \underbrace{\mathbf{f}(\mathbf{x}_k)^T \mathbf{f}(\mathbf{x}_j)}_{=K_{k,j}} \alpha_j \\
 \underbrace{\hspace{10em}}_{=K_{i,j}^2}
 \end{aligned}$$

Problem 4

Solution (cont'd)

And substituting again $\mathbf{c} = \sum_{j=1}^n \alpha_j \mathbf{f}(\mathbf{x}_j)$, the right-hand side becomes

$$\lambda \mathbf{f}(\mathbf{x}_i)^T \sum_{j=1}^n \alpha_j \mathbf{f}(\mathbf{x}_j) = \lambda \sum_{j=1}^n \underbrace{\mathbf{f}(\mathbf{x}_i)^T \mathbf{f}(\mathbf{x}_j)}_{=K_{ij}} \alpha_j$$

These hold for any $i = 1, \dots, n$

Thus, in vector form (and multiplying both sides by n)

$$K^2 \boldsymbol{\alpha} = n\lambda K \boldsymbol{\alpha},$$

which is [Equation \(1.15\)](#)